

Reviewer Pack — Minimal Representation Rank of Tensor Powers and Monotone Ci...

Entry 40c3367facbb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 20:53:34 UTC

Minimal Representation Rank of Tensor Powers and Monotone Circuit Lower Bounds

Entry ID: 40c3367facbb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 20:53:34 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory: Tensor Products **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

[‘For all $n \geq 2$, the minimal representation rank $\rho(T)$ of the tensor power T_n of the symmetric group S_3 acting on $\{1, \dots, n\}$ is upper bounded by $\rho(\det_m^{O(1)})$ for all $m < n^{\{1.5\}}$, where $\det_m$ is the determinant function over F_2 .’, ‘Furthermore, there exists a constructive mapping that transforms an instance of the k -clique problem into an element of S_3 -representations such that the representation rank $\rho(f)$ of the homogeneous polynomial f corresponds to the circuit complexity of the instance.’]

Rationale (proposer’s reasoning):

[‘Tensor products are fundamental in representation theory and can be used to encode complex structures. By linking them with monotone circuit complexity, we aim to expose a new perspective on computational hardness.’, ‘The conjecture proposes that the representation rank of tensor powers can provide lower bounds for monotone circuits, which is a novel application of representation theory.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3e187f1ba43bf2bc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all $n \leq 40$, the minimal representation rank $\rho(T_n)$ of the tensor power T_n of S_3 acting on $\{1, \dots, n\}$ is less than or equal to $\rho(\det_m^{O(1)})$ for all $m < n^{\{1.5\}}$, with at least 30 seeds ensuring that this condition holds for 80% of the tested instances.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tensor products AND monotone circuit lower bounds - symmetric group S₃ AND representation rank tensor power AND circuit complexity - k-clique problem AND transformation INTO S₃-representations AND polynomial representation rank

Top relevant hits considered: - [<http://arxiv.org/abs/2312.00421v1>] A Semi-Tensor Product based Circuit Simulation for SAT-sweeping - [<http://arxiv.org/abs/2601.13117v2>] The Case for Cardinality Lower Bounds - [<http://arxiv.org/abs/1302.3308v1>] Arithmetic Circuit Lower Bounds via MaxRank - [<http://arxiv.org/abs/hep-ph/0610012v1>] Tevatron-for-LHC Report of the QCD Working Group - [<http://arxiv.org/abs/0902.2349v1>] Defect of characters of the symmetric group - [<http://arxiv.org/abs/1908.06409v1>] Schur multipliers of special p-groups of rank 2 - [<http://arxiv.org/abs/nlin/0209047v1>] Darboux transformation for classical acoustic spectral problem - [<http://arxiv.org/abs/2501.18069v1>] Local Basis Transformation to Mitigate Negative Sign Problems

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiplication(A, B):
    if len(A[0]) != len(B):
        raise ValueError("Matrix multiplication not possible")

    rows_A = len(A)
    cols_A = len(A[0])
    cols_B = len(B[0])
```

```

C = [[Fraction(0) for _ in range(cols_B)] for _ in range(rows_A)]

for i in range(rows_A):
    for j in range(cols_B):
        for k in range(cols_A):
            C[i][j] += A[i][k] * B[k][j]

return C

def identity_matrix(n):
    I = [[Fraction(1) if i == j else Fraction(0) for j in range(n)] for i in range(n)]
    return I

def determinant(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]

    det = Fraction(0)
    sign = Fraction(1)

    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
        det += sign * matrix[0][j] * determinant(submatrix)
        sign *= -Fraction(1)

    return det

def minimal_representation_rank(n):
    S3 = [
        [Fraction(1), Fraction(0), Fraction(0)],
        [Fraction(0), Fraction(1), Fraction(0)],
        [Fraction(0), Fraction(0), Fraction(1)],

        [Fraction(0), Fraction(1), Fraction(0)],
        [Fraction(1), Fraction(0), Fraction(0)],
        [Fraction(0), Fraction(0), Fraction(1)],

        [Fraction(0), Fraction(0), Fraction(1)],
        [Fraction(0), Fraction(1), Fraction(0)],
        [Fraction(1), Fraction(0), Fraction(0)]
    ]

    T_n = identity_matrix(n)
    for _ in range(n):
        T_n = matrix_multiplication(T_n, S3)

    det_m = determinant([[Fraction(1) if i == j else Fraction(0) for j in range(m)] for i in range(m)])
    rho_det_m = len([x for x in det_m.numerator.as_integer_ratio()[1].primefactors()]) + 1

    return len([x for x in T_n[0][0].as_integer_ratio()[1].primefactors()]) + 1

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(2, 40)

```

```

rho_T_n = minimal_representation_rank(n)

m = math.floor(n ** 1.5)
det_m = determinant([[Fraction(1) if i == j else Fraction(0) for j in range(m)] for i in range(m)])
rho_det_m = len([x for x in det_m.numerator.as_integer_ratio()[1].primefactors()]) + 1

metric_value = rho_T_n <= rho_det_m
conjecture_holds = metric_value
counterexample = "" if conjecture_holds else f"rho(T_{n})={rho_T_n}, rho(det_{m})={rho_det_m}"

return {
    "metric_name": "minimal_representation_rank",
    "metric_value": float(metric_value),
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys

    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...{result}...}}")
        results.append(result)

    total_metric_value = sum(r["metric_value"] for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={total_metric_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={total_metric_value / len(results)} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[seeds.index(first_failing_seed)][\"counterexample\"]}\"")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ceb158bf.py", line 115, in <module>
    result = run_trial(seed)
            ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ceb158bf.py", line 90, in run_trial
    rho_T_n = minimal_representation_rank(n)
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ceb158bf.py", line 79, in minimal_rep

```

